# Host your Python agents on Microsoft Foundry

Apr 27: **Microsoft Agent Framework**

Apr 29: **LangChain and LangGraph**

Apr 30: **Quality and safety evals**

Register at aka.ms/AgentsOnFoundry/series

Microsoft

# Host your agents on Foundry

## Microsoft Agent Framework

aka.ms/foundryhosted/slides/agentframework

Pamela Fox

Python Cloud Advocate

www.pamelafox.org

# Today we'll cover…

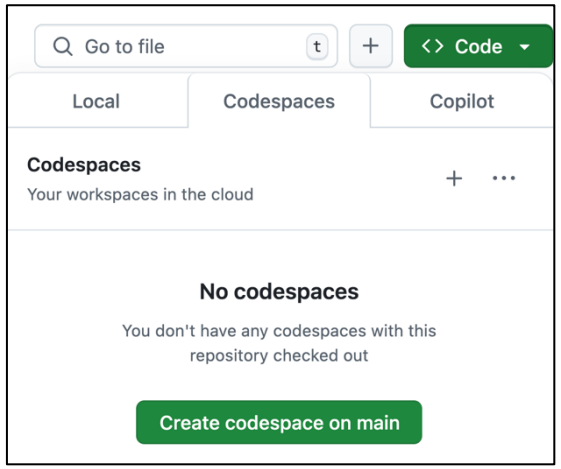
- Agents 101
- Building agents with Microsoft Agent Framework
- Foundry models for inference
- Foundry IQ for knowledge retrieval
- Foundry tools for cloud-hosted agentic tools
- Hosting agents on Foundry Agent Service
- Building multi-agent workflows
- Hosting workflows on Foundry Agent Service

# Want the code?

1. Open this GitHub repository:

[aka.ms/foundry-hosted-agentframework-demos](aka.ms/foundry-hosted-agentframework-demos)

2. Use "Code" button to create a GitHub Codespace:

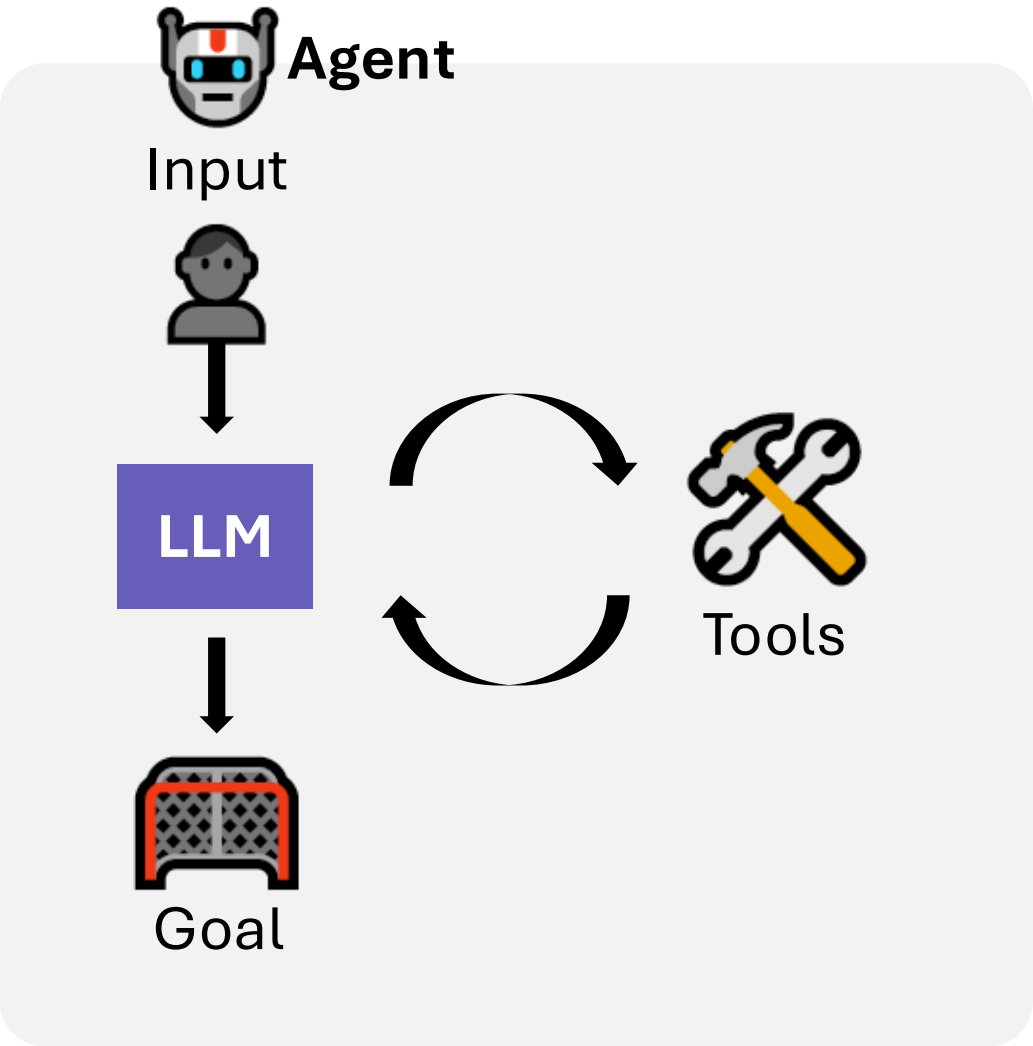



3. Wait a few minutes for Codespace to start up ⏱️

⚠️ Most code samples require deployment and an Azure account.

# Agents 101

# What is an agent?



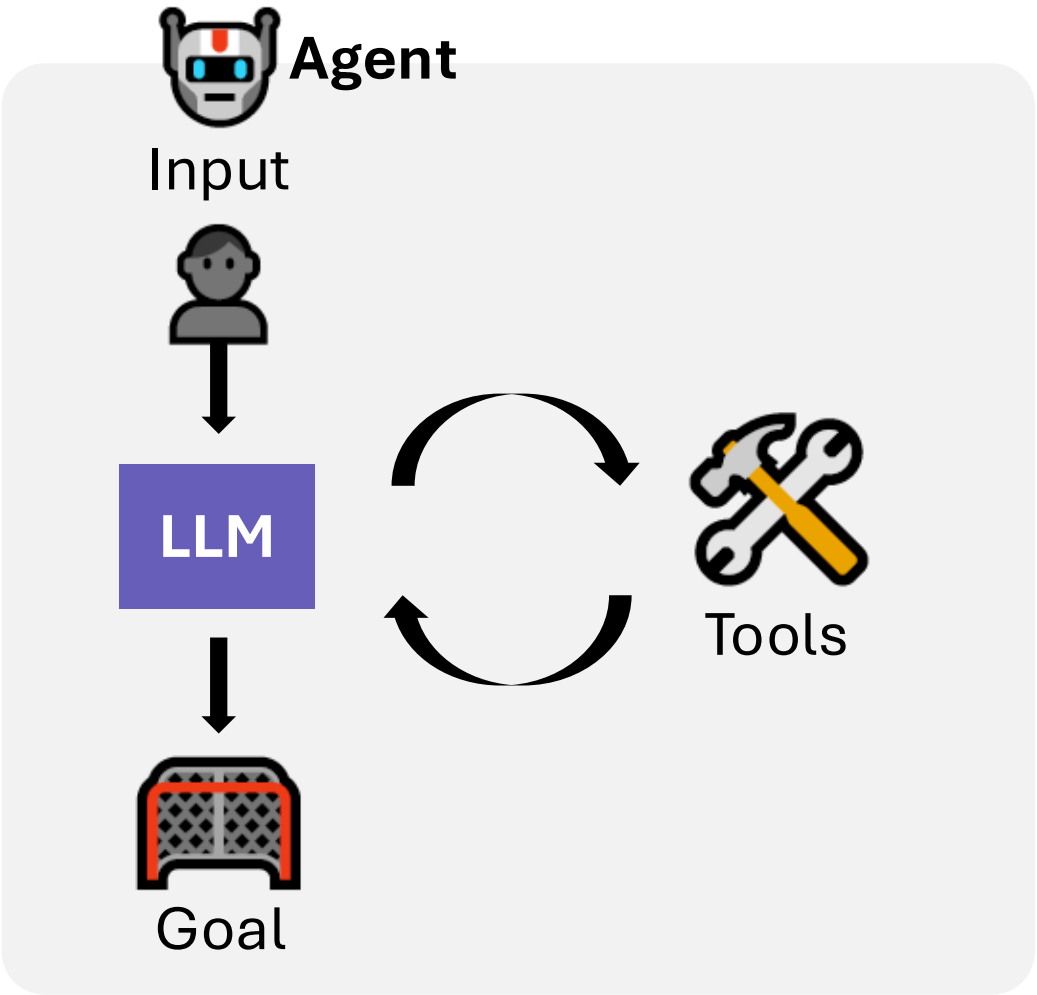

An **AI agent** uses an **LLM** to run **tools** in a loop to achieve a **goal**.

Agents are often augmented by:

- Context
- Memory
- Planning
- Humans

Dive into advanced agentic features in our previous Python + Agents series:
aka.ms/pythonagents/rewatch

# The agentic loop in action




The agent keeps calling tools until goal is met:

# Python AI agent frameworks

A framework makes it easier to handle tool calling and context management:

| Framework | Description |
|---|---|
| **agent-framework** | A framework from Microsoft with support for agentic patterns and full integration with Azure offerings. (Successor to autogen and semantic-kernel) |
| **langchain v1** | An agent-centric framework built on top of Langgraph, with optional Langsmith monitoring |
| **pydantic-ai** | A flexible framework designed for type safety and observability (Logfire/OTel), from the creators of Pydantic |

There are many frameworks, but these are popular Python frameworks.

# Microsoft Agent Framework

Python .NET

Open-source engine for building and orchestrating intelligent AI agents and workflows

Based on two popular frameworks from Microsoft:

## Semantic Kernel

Full SDK designed to build AI agents with ease, excellent for single agents and can be extended for multi-agents with integrations to AutoGen

+

## AutoGen

Powerful multi-agent research framework with pre-built conversation orchestration patterns for handling complex agent systems

Agents • Orchestration • Memory • State • Cloud-agnostic • Enterprise-ready

aka.ms/AgentFramework

# Installing Agent Framework for Python

Either install the all-in-one package:

```
>> pip install agent-framework
```

OR install more selectively. Always install core:

```
>> pip install agent-framework-core
```

Then install only the packages you need, choosing from...

```
agent-framework-a2a
agent-framework-ag-ui
agent-framework-anthropic
agent-framework-azure-ai-search
agent-framework-azure-cosmos
agent-framework-azurefunctions
agent-framework-bedrock
agent-framework-chatkit
```

```
agent-framework-claude
agent-framework-copilotstudio
agent-framework-declarative
agent-framework-devui
agent-framework-durabletask
agent-framework-foundry
agent-framework-foundry-local
agent-framework-github-copilot
```

```
agent-framework-lab
agent-framework-mem0
agent-framework-ollama
agent-framework-openai
agent-framework-orchestrations
agent-framework-purview
agent-framework-redis
```

📌 Always pin your versions and use a lock file!

📎 See example: [pyproject.toml]

# A simple local agent with tools

agent-framework

```python
from agent_framework import Agent, tool
from agent_framework.openai import OpenAIChatClient

client = OpenAIChatClient(
    base_url="http://localhost:11434/v1/",
    model="qwen3.5:4b",
    api_key="no-key-needed")


@tool
def get_enrollment_deadline_info() -> dict:
  """Return enrollment timeline details for health insurance plans."""
  return {"enrollment_opens": "2026-11-11", "enrollment_closes": "2026-11-30"}

agent = Agent(client=client,
  instructions=f"You're an HR helper. Today's date is {date.today().isoformat()}. ",
  tools=[get_enrollment_deadline_info])


response = await agent.run("When does benefits enrollment open?")
print(response.text)
```

Full example: [agents/stage0_local_model.py]

# 🤖 Building agents on Foundry

# Microsoft Foundry

The end-to-end platform to design, customize, and deploy AI-driven applications at scale.

### Models

Access frontier models like Opus 4.6 and GPT-5.4.

### IQ

Ground LLMs in domain data, SharePoint, web search, and more.

### Agent Service

Host agents with memory, tools, observability, and evaluations.

### Tools

Connect to 1,400+ built-in service connections and MCP tools.

### Machine Learning

Train and deploy custom ML models for specific business needs.

**Control Plane**

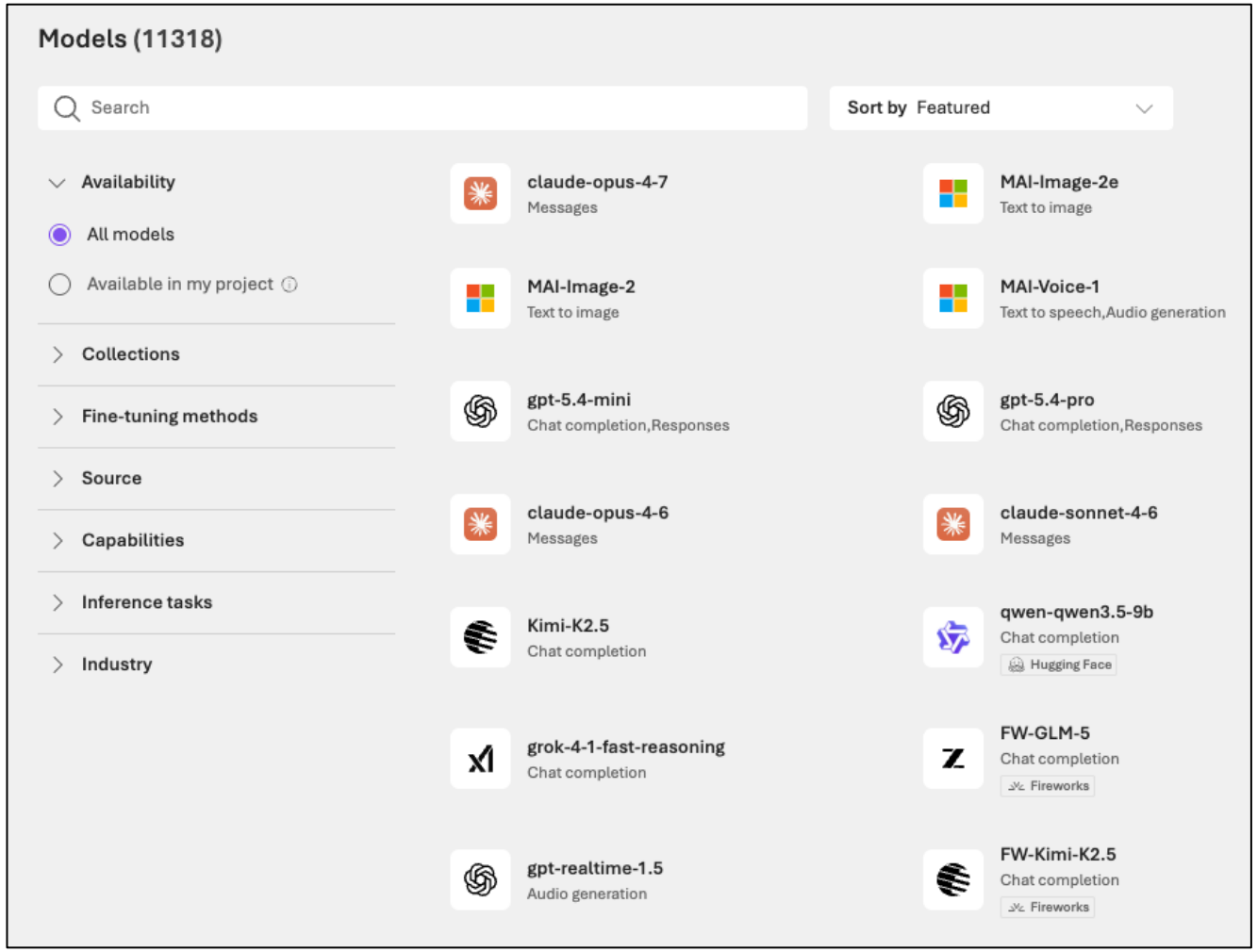
**Leverage a complete signals management layer with Microsoft Security integrations**

Microsoft Agent 365 | Microsoft Defender | Microsoft Purview | Microsoft Entra

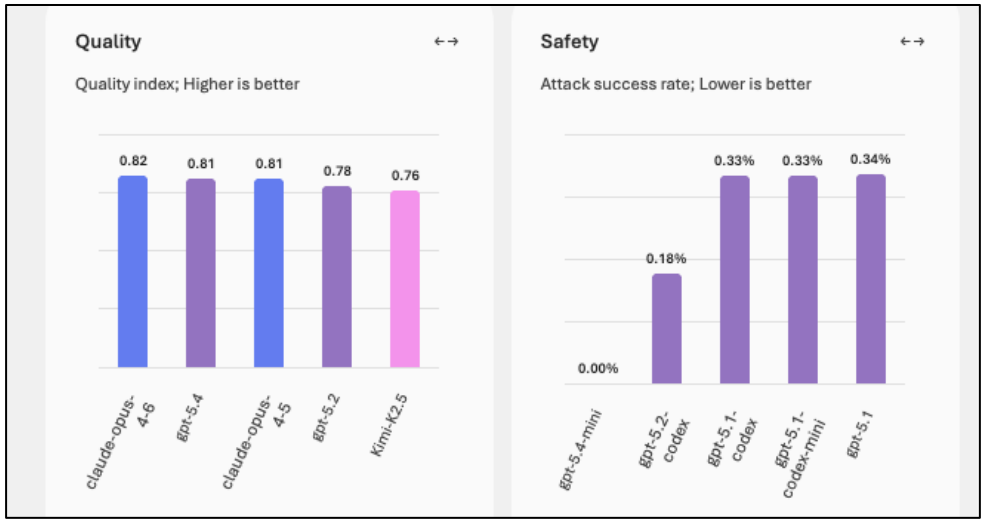
**Security, compliance, and governance**

# Foundry Models

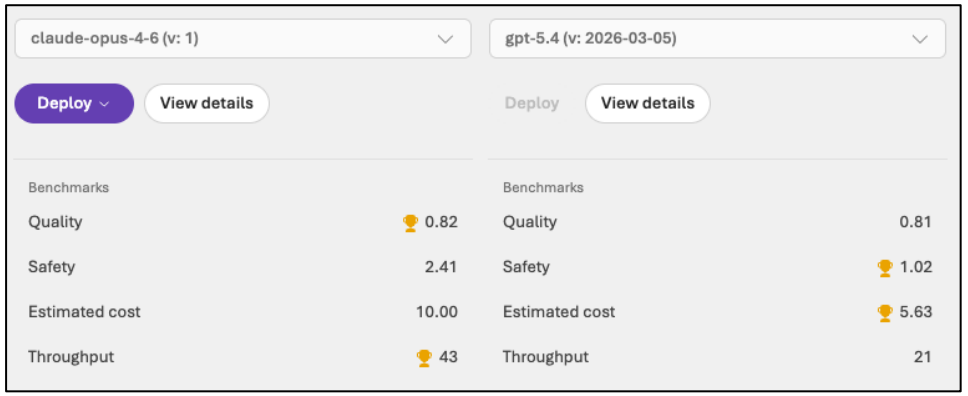
11,000+ models, both open-source and frontier models.




See overall metrics on leaderboards:




Compare models side-by-side:



# An agent with Foundry models

agent-framework

```python
client = OpenAIChatClient(
  base_url=f"{os.environ['AZURE_OPENAI_ENDPOINT']}/openai/v1/",
  api_key=token_provider,
  model=os.environ["AZURE_AI_MODEL_DEPLOYMENT_NAME"])
```

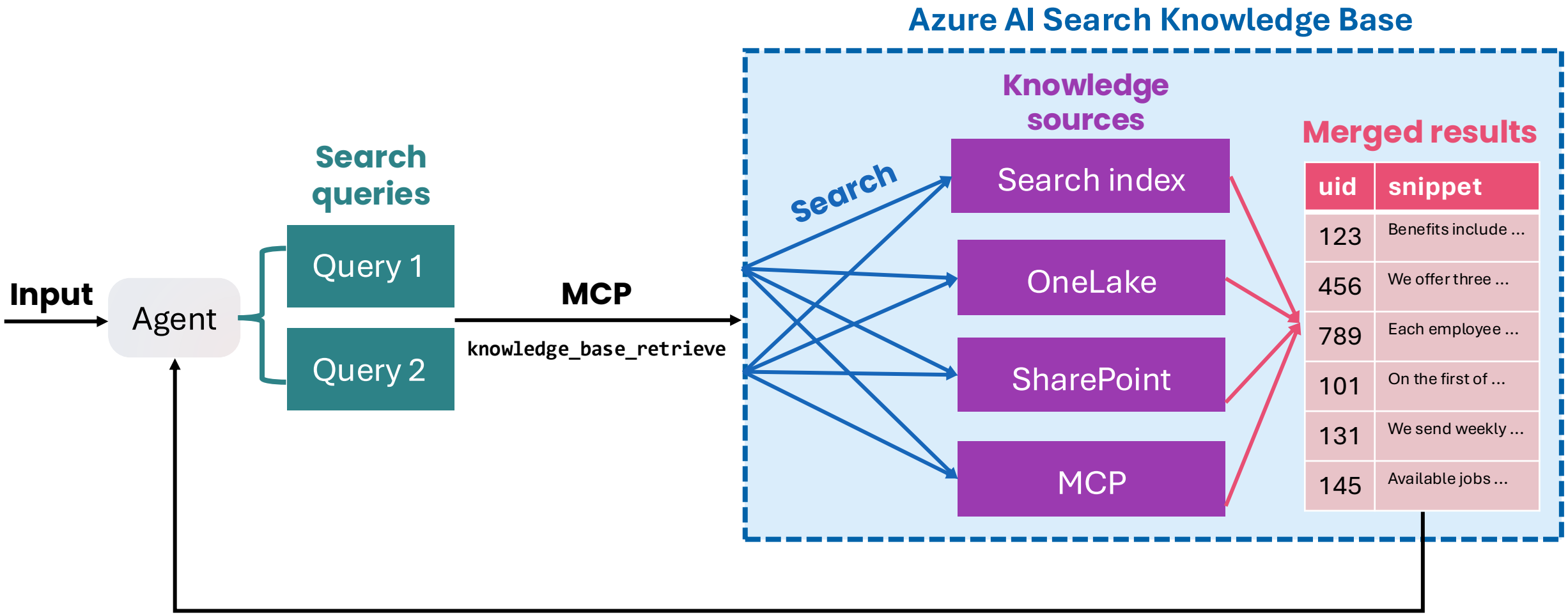
Only the client changes

```python
@tool
def get_enrollment_deadline_info() -> dict:
"""Return enrollment timeline details for health insurance plans."""
 return {"enrollment_opens": "2026-11-11", "enrollment_closes": "2026-11-30"}

agent = Agent(client=client,
  instructions= f"You're an HR helper. Today's date is {date.today().isoformat()}.",
  tools=[get_enrollment_deadline_info])


response = await agent.run("When does benefits enrollment open?")
print(response.text)
```

Full example: agents/stage1_foundry_model.py

# Foundry IQ

Generate unified context using agentic retrieval across multiple knowledge sources (indexed and remote)



# Adding Foundry IQ as MCP tool

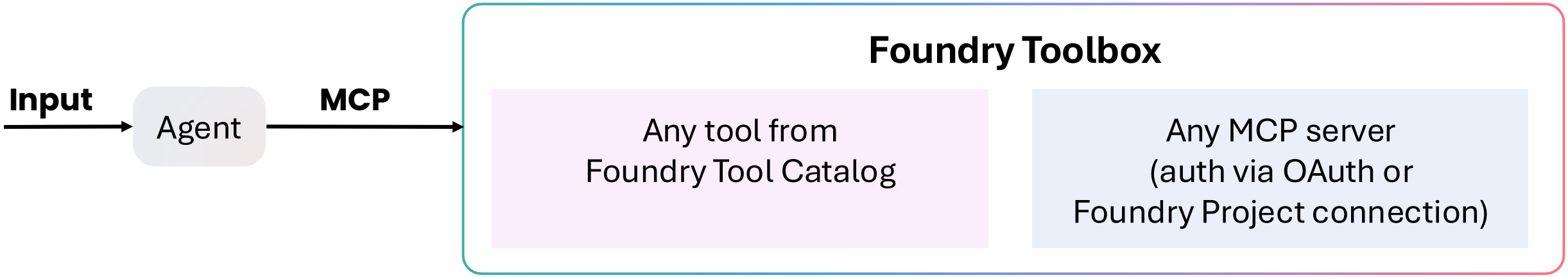
agent-framework

```
mcp_url = (f"{AZURE_AI_SEARCH_SERVICE_ENDPOINT}/knowledgebases/"
           f"{AZURE_AI_SEARCH_KNOWLEDGE_BASE_NAME}/mcp?api-version=2025-11-01-Preview")

async with MCPStreamableHTTPTool(
  name="knowledge-base",
  url=mcp_url,
  http_client=search_http_client,
  allowed_tools=["knowledge_base_retrieve"],
  ) as kb_mcp_tool:

    agent = Agent(
      client=client,
      instructions=f"You're an HR helper. Today's date is {date.today().isoformat()}. "),
      tools=[kb_mcp_tool, get_enrollment_deadline_info])

   response = await agent.run("What PerksPlus benefits are there?")
   print(response.text)
```

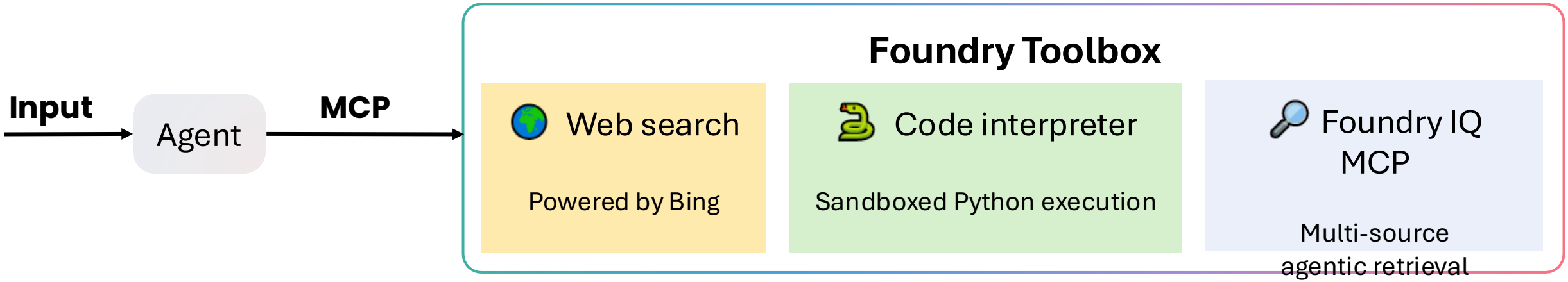
Full example: [agents/stage2_foundry_iq.py]

# Foundry Tools

Create a toolbox on Foundry composed of Foundry Tools and MCP servers, for use by any agent.




Today we'll build this toolbox:



# Creating a toolbox from code

```
tools = [
  {"type": "web_search", "name": "web_search"},
  {"type": "code_interpreter", "name": "code_interpreter"},
  {
   "type": "mcp",
   "server_label": "knowledge-base",
   "server_url": 'https://${search_service}.search.windows.net/knowledgebases/'
                 ${knowledge_base_name}/mcp?api-version=2025-11-01-preview',
   "project_connection_id": 'kb-mcp-connection',
   "allowed_tools": ["knowledge_base_retrieve"]}
]

resp = httpx.post(f"{PROJECT_ENDPOINT}/toolboxes/{TOOLBOX_NAME}/versions",
  params={"api-version": "v1"}, json={"tools": tools})

httpx.patch(f"{PROJECT_ENDPOINT}/toolboxes/{TOOLBOX_NAME}",
  params={"api-version": "v1"}, json={"default_version": resp.json()["version"]})
```

Full code: [create-toolbox.py]

KB Connection made in: [azure_ai_search.bicep]

# Equip agent with toolbox

agent-framework

```python
toolbox_token_provider = get_bearer_token_provider(credential, "https://ai.azure.com/.default")
toolbox_http_client = httpx.AsyncClient(
  auth=ToolboxAuth(toolbox_token_provider),
  headers={"Foundry-Features": "Toolboxes=V1Preview"},
  timeout=120.0)

toolbox_mcp_tool = MCPStreamableHTTPTool(
  name="toolbox",
  url=f"{PROJECT_ENDPOINT}/toolboxes/{TOOLBOX_NAME}/mcp?api-version=v1",
  http_client=toolbox_http_client)

async with toolbox_mcp_tool:
    agent = Agent(
      client=client,
      tools=[
        get_enrollment_deadline_info,
        toolbox_mcp_tool
      ])
```

Full example: agents/stage3_foundry_toolbox.py

# Foundry Agent Service

Host your agents with production-grade security, reliability, and governance.

## Prompt Agents
Define instructions and tools

## Hosted Agents
Use your favorite agent framework

## Foundry Tools

## Foundry IQ

## Foundry Memory
Managed Memory
Managed Conversations
BYO-Memory Store

## Foundry Models

## Foundry Control Plane

- Controls
- Observability
- Security
- Fleet-wide Operations

Microsoft Agent 365

Microsoft Defender

Microsoft Entra

Microsoft Purview

# Hosted MAF Agents in Foundry Agent Service

Serve any agent using the Responses API or more generic invocations API.

**Foundry Agent Service**

Identity · Endpoint · State Scaling · Observability

**Container (in Azure Container Registry)**

**Responses API Adapter** `/agents/{name}/endpoint/protocols/openai/responses`

**Your code**

.NET

Java

JS

# Hosted MAF Agents in Foundry Agent Service

Microsoft Agent Framework provides a built-in Adapter – just wrap your agent and run.

## Foundry Agent Service

### Container (in Azure Container Registry)

`Dockerfile` `agent.yaml`

#### Responses API Adapter

```
from agent_framework_foundry_hosting import ResponsesHostServer
```

##### Your code

```
agent = Agent(
  client=FoundryChatClient(project_endpoint, model, credential),
  tools=[...])
```

```
server = ResponsesHostServer(agent)
server.run()
```

# A Foundry hosted MAF agent

agent-framework

```python
from agent_framework import Agent, MCPStreamableHTTPTool
from agent_framework.foundry import FoundryChatClient
from agent_framework_foundry_hosting import ResponsesHostServer


client = FoundryChatClient(
  project_endpoint=PROJECT_ENDPOINT, model=MODEL_DEPLOYMENT_NAME, credential=credential)

agent = Agent(
  client=client,
  name="InternalHRHelper",
  instructions="You are an internal HR helper.",
  tools=[get_enrollment_deadline_info, get_current_date, toolbox_mcp_tool],
  default_options={"store": False})

server = ResponsesHostServer(agent)
server.run()
```

Full example: [agents/stage4_foundry_hosted.py]

# Deploy agent to Foundry with azd

```
>> azd auth login
>> azd up
```

What this does:

1. Provision infrastructure (AI project, model)
2. Builds image from Dockerfile (using ACR remote build)
3. Push image to Azure Container Registry
4. Deploys new agent version to Foundry

All based off these project files:

azure.yaml

```yaml
name: ai-foundry-starter-basic
services:
 hosted-agentframework-agent:
  project: ./agents
   host: azure.ai.agent
   language: docker
   docker:
    remoteBuild: true
   config:
    startupCommand: python
stage4_foundry_hosted.py
```

Dockerfile

```dockerfile
FROM python:3.12-slim
WORKDIR /app
COPY --from=ghcr.io/astral-sh/uv:latest /uv /bin/uv
COPY pyproject.toml uv.lock ./
RUN uv sync --locked --no-dev
COPY stage4_foundry_hosted.py .
ENV PATH="/app/.venv/bin:$PATH"

EXPOSE 8088

CMD ["python", "-u", "stage4_foundry_hosted.py"]
```

agent.yaml

```yaml
kind: hosted
name: hosted-agentframework-agent
protocols:
    - protocol: responses
      version: 1.0.0
resources:
  cpu: '0.25'
  memory: '0.5Gi'
environment_variables:
    ...
```

# Interact with Foundry agent in playground

Microsoft Foundry / ai-project-pf-foundryhosted-agentframework

Search with AI (⌘K) | New Foundry | Home | Discover | Build | Operate | Docs

- Agents
- Models
- Fine-tune
- Tools
- Knowledge
- Data
- Evaluations
- Guardrails

## hosted-agentframework-agent

v7 saved 4/20/2026 3:26 PM | Active | Publish

Playground | Traces | Monitor | Evaluation

Chat | Code | Metrics

**Examples of what's covered (not exhaustive):**

- Gym memberships
- Personal training sessions
- Yoga and Pilates classes
- Fitness equipment purchases
- Sports team fees
- Health retreats and spas
- Outdoor adventure activities (e.g., rock climbing, hiking, kayaking)
- Group fitness classes (e.g., dance, martial arts, cycling)
- Virtual fitness programs (online yoga/workout classes)
- Lessons/experiences like **skiing/snowboarding**, **scuba diving**, **surfing**, and **horseback riding** lessons

**Not covered (examples):**

- Non-fitness-related expenses
- Medical treatments/procedures
- Travel expenses (unless related to a fitness program)
- Food and supplements

Traces

Message the agent...

AI-generated content may be incorrect

### Agent info

**Entra agent identity**
36ab37c4-30ed-4b45...

**Entra agent blueprint**
hosted-agentframew...

**Code asset**
crnivdj7gkpbmk2.azurecr.io/ai-foundry-starter-basic/...

**Kind**
hosted

**Protocol**
responses

Log stream | Files | Session: 0e2950ed0c942af2dacd...

Type to filter logs...

07:10:58.883 [stream] stderr 2026-04-21 14:10:58,883 INFO agent_framework: {'role': 'system', 'parts': [{'type': 'text', 'content': 'You are an internal HR helper focused on employee benefits and company information.\n Use the knowledge base tool to answer questions and ground all answers in provided context.\n You

# Interact with Foundry agent from code

You can make direct HTTP calls to the Responses API endpoint:

```
curl -X POST "$BASE_URL/agents/$AGENT_NAME/endpoint/protocols/openai/responses?api-version=$API_VERSION" \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{ "input": "Hello! What can you do?", "store": true }'
```

Or use the azure-ai-projects package to send responses via an OpenAI client:

```
from azure.ai.projects import AIProjectClient

project = AIProjectClient(
  endpoint=os.environ["FOUNDRY_PROJECT_ENDPOINT"],
  credential=DefaultAzureCredential(),
  allow_preview=True)

openai_client = project.get_openai_client(agent_name="hosted-agentframework-agent")
response = openai_client.responses.create(input="What PerksPlus benefits are there?")
print(response.output_text)
```

🔗 Full example: [agents/call_foundry_hosted.py]

# Local development loop

Terminal 1

```
>> azd ai agent run

Using startup command: python stage4_foundry_hosted.py
Installing dependencies (pyproject.toml)...
  ✓ Dependencies installed (pyproject.toml)
Starting agent on http://localhost:8088
```

Terminal 2

```
>> azd ai agent invoke --local "What benefits are there?"

Target:       localhost:8088 (local)
Message:      "What PerksPlus benefits are there?"
Session:      6d872d38-a8e5-40bb-9813-34fb272c1785
Conversation: d47ecf0f-c2d6-485a-b628-561ec180bc92

[local] PerksPlus is Zava’s health & wellness reimbursement program. It lets employees
**expense up to $1,000** for eligible fitness/wellness-related items and activities.
```

# 👀 Observability

# Observability with OpenTelemetry (OTel)

OTel standardizes how applications emit traces, metrics, and logs — making debugging and performance analysis consistent across languages and vendors.

## Traces

```
0ms                                30ms
parent-span
└→ child-span
```

Operations composed of **spans** that show how a request moves through services, including timing, dependencies, and context propagation.”

## Metrics

```
_/‾‾ cpu_usage: 30%
‾‾\__ latency_p95: 43ms
_/‾\__ errors: 0.2%
```

Numeric measurements such as CPU usage, request counts, latency, error rates, or any custom app metric.

## Logs

```
INFO: 2025-12-05: Server running
      {"region": "westus"}
ERROR: 2025-12-05: User not found
      {"user_id": 123}
```

Structured log records with message, severity, time stamp, and contextual attributes.

# Using OpenTelemetry with Agent Framework

Agent framework has built-in support for emitting OpenTelemetry traces:

```
from agent_framework.observability import enable_instrumentation

enable_instrumentation(enable_sensitive_data=True)
```

🔗 Full example: [agents/stage4_foundry_hosted.py]

Agent framework uses the standard "gen_ai" span attribute names and values for agent executions, LLM calls, and tool calls. Example tool call:

**Span:** `execute_tool get_weather`

| attribute name | value |
|---|---|
| **gen_ai.operation.name** | execute_tool |
| **gen_ai.tool.name** | get_weather |
| **gen_ai.tool.call.arguments** | {"city": "Seattle"} |
| **gen_ai.tool.call.result** | {"temperature": 72, "description": "Sunny"} |
| **gen_ai.tool.description** | Returns weather data for a given city |

# Exporting OTel to App Insights

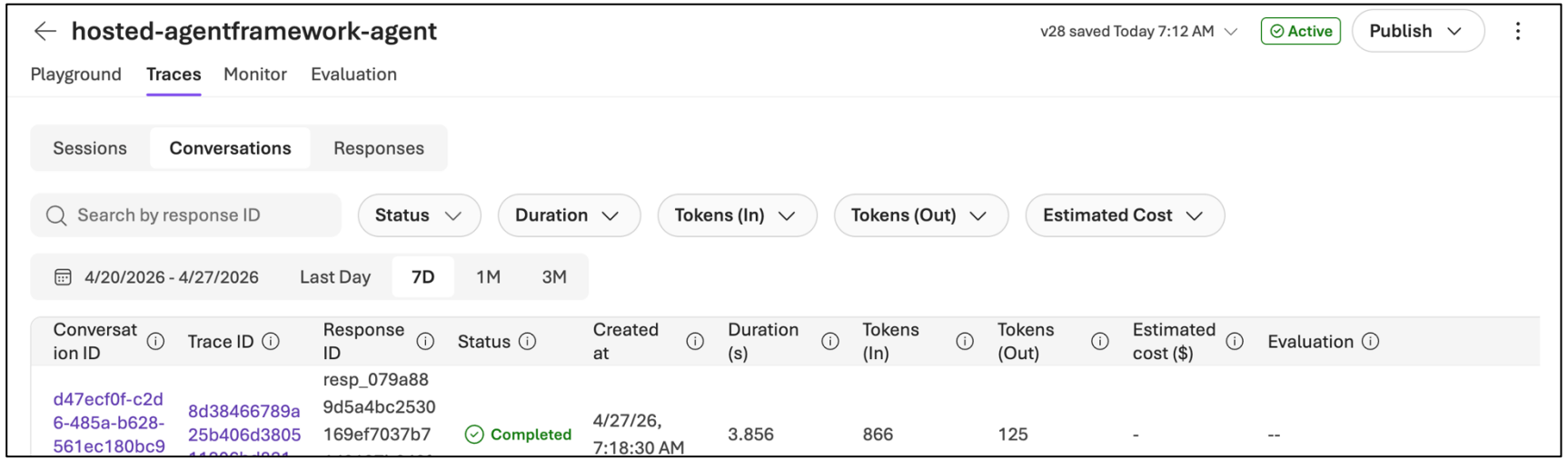
When the Foundry Project has an App Insights connection, Foundry agent service will auto-inject `APPLICATIONINSIGHTS_CONNECTION_STRING` environment variable.

Then `ResponsesHostServer` configures the export to App Insights:

```
from agent_framework_foundry_hosting import ResponsesHostServer
from agent_framework.observability import enable_instrumentation

enable_instrumentation(enable_sensitive_data=True)

agent = Agent(...)

server = ResponsesHostServer(agent)
server.run()
```

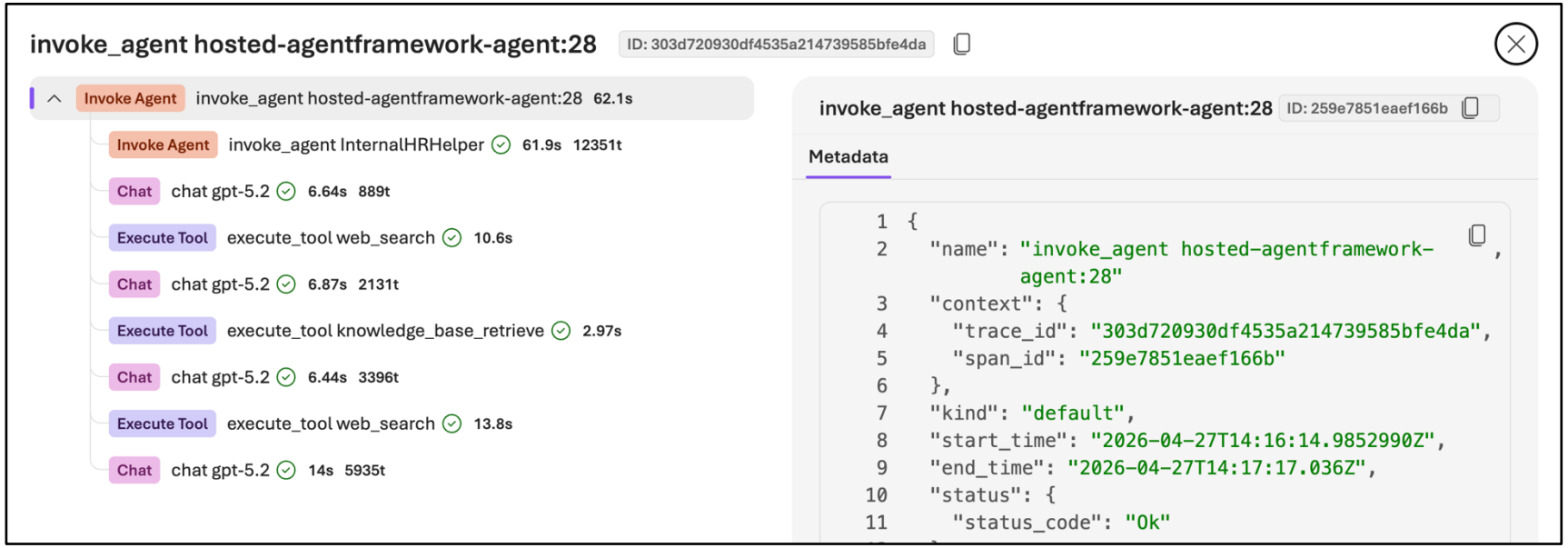
🔗 Full example: [agents/stage4_foundry_hosted.py]

# Viewing agent traces in Foundry




From Foundry agent, select :

Traces > Conversations




See spans for:

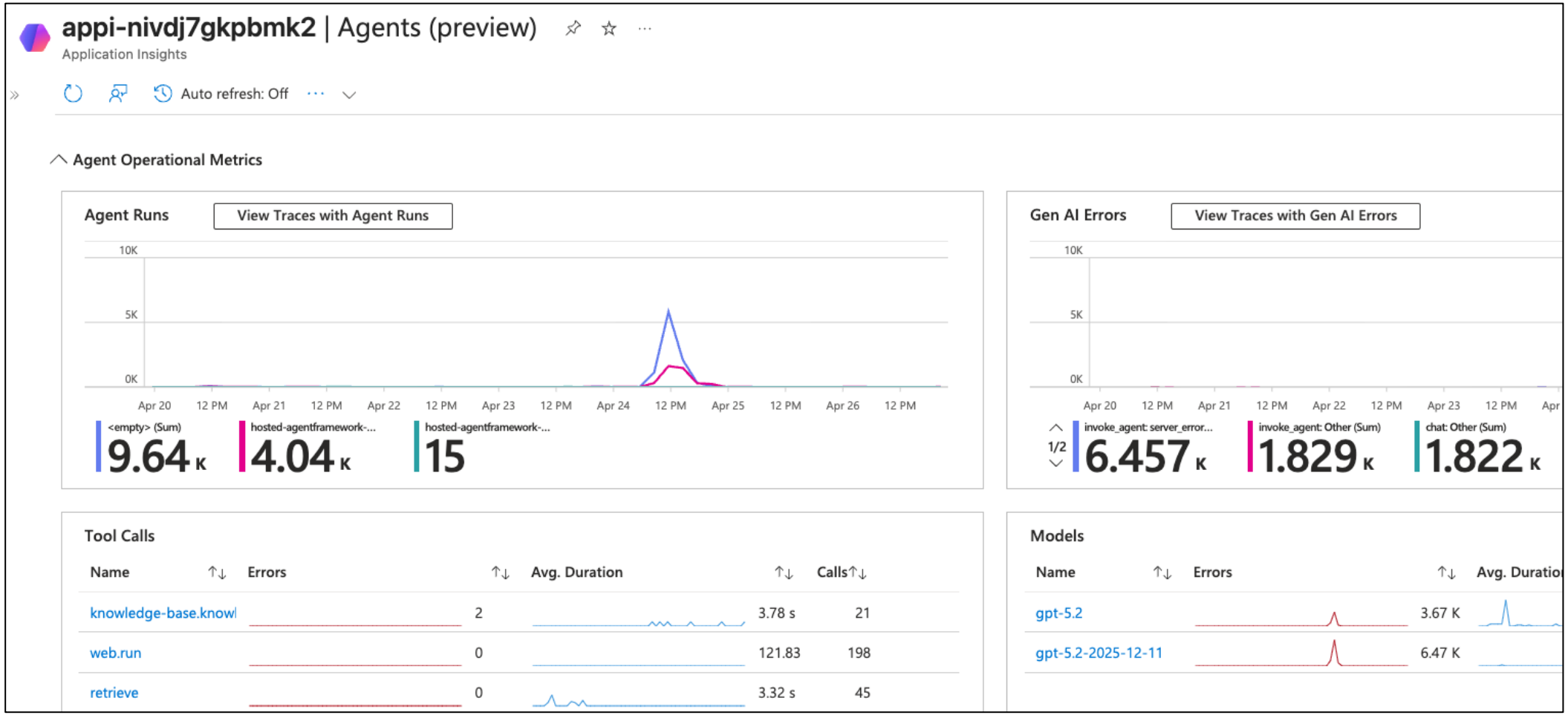
agent invocations,

LLM calls,

tool calls

# Viewing agent traces in App Insights

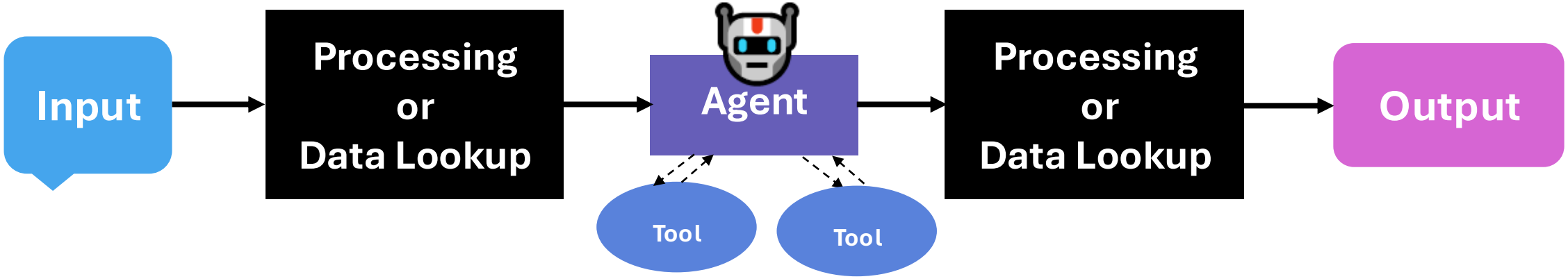
From Foundry agent, select Monitor > Open in Azure Monitor:



# Multi-agent workflows

# What's a workflow?

An agentic workflow is any flow that involves an agent at some point, typically to handle decision making or answer synthesis.



# Building workflows in Agent Framework

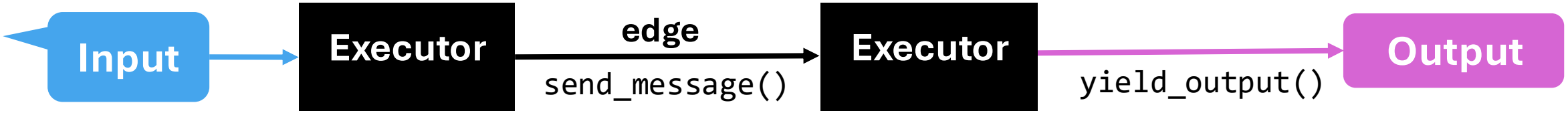
A workflow is a graph with `Executor` nodes and edges between them:

Each executor subclass defines handler methods that **receive messages** and may **send messages to next executor** and/or **yield outputs.**

Dive deeper into MAF workflows in our previous Python + Agents series: (Week 2)
📺 aka.ms/pythonagents/rewatch

# A simple workflow

agent-framework



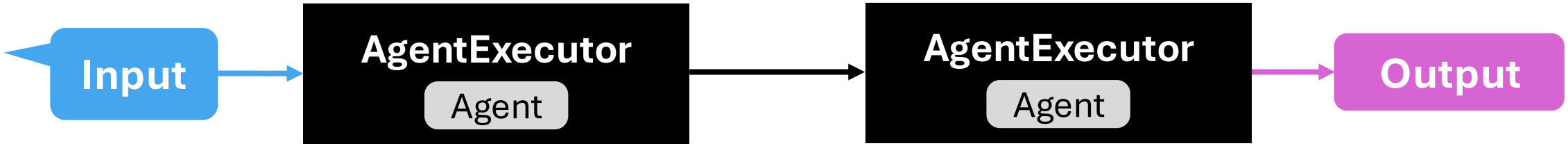

```python
class UpperCase(Executor):
  @handler
  async def to_upper_case(self, text: str, ctx: WorkflowContext[str]):
    await ctx.send_message(text.upper())

class ReverseText(Executor):
  @handler
  async def reverse(self, text: str, ctx: WorkflowContext[str, str]):
    await ctx.yield_output(text[::-1])

upper = UpperCase()
reverse = ReverseText()
workflow = WorkflowBuilder(start_executor=upper).add_edge(upper, reverse).build()
events = await workflow.run("hello world")
print(f"Output: {events.get_outputs()}")
```

Full example: [workflows/stage1_simple_executors.py]

# Using agents inside workflows

agent-framework

Every `Agent` instance can easily be used as an `Executor` in a workflow:



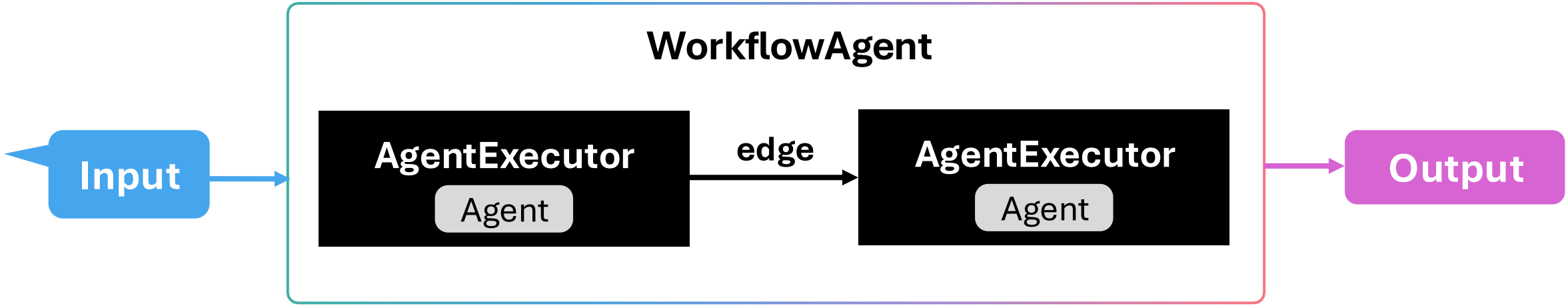

```
writer = Agent(client=client, name="Writer",
  instructions="You are a concise content writer. ")
formatter = Agent(client=client, name="Formatter",
  instructions="Format text with emojis and Markdown.")

writer_executor = AgentExecutor(writer, context_mode="last_agent")
formatter_executor = AgentExecutor(formatter, context_mode="last_agent")

workflow = (WorkflowBuilder(
  start_executor=writer_executor,
  output_executors=[formatter_executor])
  .add_edge(writer_executor, formatter_executor)
  .build())
```

Full example: workflows/stage2_agent_executors.py

# Using workflow as an agent

agent-framework

A workflow can be turned into an agent for compatibility with other systems.




```
workflow_agent = workflow.as_agent(name="Content Pipeline")
```

🔗 Full example: workflows/stage3_as_agent.py

# A Foundry hosted MAF workflow

agent-framework

```python
from agent_framework.observability import enable_instrumentation
from agent_framework_foundry_hosting import ResponsesHostServer

enable_instrumentation(enable_sensitive_data=True)

# setup agents and executors...

workflow_agent = (
  WorkflowBuilder(
    start_executor=writer_executor,
    output_executors=[format_executor])
  .add_edge(writer_executor, format_executor)
  .build()
  .as_agent()
)

server = ResponsesHostServer(workflow_agent)
server.run()
```

Full example: workflows/stage4_foundry_hosted_as_agent.py

# Next steps

Register:
https://aka.ms/AgentsOnFoundry/series

Watch past recordings:
aka.ms/foundryhosted/resources

Join office hours after each session in Discord:
aka.ms/pythonai/oh

Apr 27: **Microsoft Agent Framework**

Apr 29: **LangChain and LangGraph**

Apr 30: **Quality and safety evals**